VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

E-Commerce Order Flow

A9512 – Database Management System Laboratory

Submitted by

P Ashmith	25881A0502
E Hanish	25881A0512
P Rithvik	25881A0539
D Sai Charan	25881A0543

Course Instructor

Ms. Shaik Shahanaz
Assistant Professor
Department of CSE

Department of Computer Science and Engineering
Vardhaman College of Engineering, Hyderabad

October 2026

VARDHAMAN
COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,**
Approved by **AICTE,** Accredited by **NAAC** with **A**++ Grade
Kacharam, Shamshabad, Hyderabad- 501 218,
Telangana, India.

26 Years of Vibrant Vardhaman

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **"E-Commerce Order Flow"** is carried out by **"P Ashmith" and "E Hanish" and "P Rithvik" and "D Sai Charan" with** Roll Numbers **"25881A0502" and "25881A0512" and "25881A0539" and "25881A0543"** towards **A9512 – Database Management System Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2026-27.

Course Facilitator	Academic Head	Head of the Department
Ms. Shaik Shahanaz	**Dr. R. Karthikeyan**	**Prof. Y. Vijayalata**
Assistant Professor, CSE	Professor, CSE	HoD, CSE

Contents

Abstract **1**

1 Introduction **2**
1.1 Background . . . 2
1.2 Need for the System . . . 2
1.3 Scope . . . 2
1.4 Organization of the Report . . . 2

2 Problem Analysis **3**
2.1 Problem Statement . . . 3
2.2 Users and Assumptions . . . 3
2.3 Functional Requirements . . . 3
2.4 Non-Functional Requirements . . . 3
2.5 Limitations . . . 3

3 ER Model **4**
3.1 Entities and Attributes . . . 4
3.2 Relationships . . . 4
3.3 Cardinality and Participation . . . 4
3.4 ER Diagram . . . 4

4 Relational Model **6**
4.1 Relational Schema . . . 6
4.2 Primary Keys and Foreign Keys . . . 6
4.3 Integrity Rules . . . 6

5 Normalization **7**
5.1 Functional Dependencies . . . 7
5.2 First Normal Form (1NF) . . . 7
5.3 Second Normal Form (2NF) . . . 7
5.4 Third Normal Form (3NF) . . . 7

6 SQL DDL Commands **9**
6.1 Database Creation . . . 9
6.2 Table Creation . . . 9
6.3 Constraints . . . 10
6.4 ALTER Operation . . . 10
6.5 DROP Operation . . . 10

7 SQL DML Commands **11**
7.1 INSERT . . . 11
7.2 UPDATE . . . 11
7.3 DELETE . . . 11

8 SQL Queries **12**
8.1 Basic SELECT Queries . . . 12
8.2 JOIN Operations . . . 12
8.3 Aggregate Functions . . . 12
8.4 GROUP BY and HAVING . . . 12
8.5 Subqueries . . . 13
8.6 Views . . . 13

9 Transactions and Concurrency **14**
9.1 Transaction Management . . . 14
9.2 COMMIT, ROLLBACK, and SAVEPOINT . . . 14
9.3 Concurrency Control . . . 14
9.4 Order Cancellation . . . 14

10 Stored Programs and Triggers **15**

10.1 Stored Function . . . 15
10.2 Stored Procedure . . . 15
10.3 Cursor . . . 15
10.4 Trigger . . . 16

11 File Organization and Indexing **17**
11.1 File Organization . . . 17
11.2 Indexing in the Project . . . 17
11.3 B+ Tree Indexing . . . 17
11.4 Index Selection . . . 17

12 Testing **18**
12.1 Test Cases . . . 18
12.2 Normal and Boundary Cases . . . 18
12.3 Invalid Input Testing . . . 18
12.4 Transaction Testing . . . 18
12.5 Test Evidence . . . 18

13 Reports **19**
13.1 Order Report . . . 19
13.2 Order Item Details . . . 19
13.3 Cancellation Report . . . 19
13.4 Revenue Report . . . 19
13.5 Inventory Report . . . 19
13.6 Shipping Report . . . 19

14 Execution Outputs **20**

15 Conclusion **29**

16 Mapping POs and SDGs **30**
16.1 Program Outcomes (POs) Mapping . . . 30
16.2 Sustainable Development Goals (SDGs) Mapping . . . 30

17 Future Scope **31**

Bibliography **32**

List of Tables

2.1 Users of the E-Commerce Order Flow System 3
3.1 Entities and Principal Attributes 4
3.2 Relationships, Cardinality, and Participation 4
4.1 Foreign-Key Relationships 6
5.1 Illustration of a Non-Atomic Order Record 7
5.2 Order Items in 1NF 7
5.3 Final Normalized Relations 7
12.1 Test Cases for E-Commerce Order Flow 18
14.1 Test Cases for E-Commerce Order Flow (executed results) 28
16.1 Mapping of Program Outcomes to Project Relevance 30
16.2 Mapping of SDGs to Project Relevance 30

List of Figures

3.1 ER Diagram of the E-Commerce Order Flow System 5
14.1 Execution output: Database creation (Section 6.1) 20
14.2 Execution output: Table creation (Section 6.2) 20
14.3 Execution output: Table creation, continued (Section 6.2) 20
14.4 Execution output: ALTER operation and resulting Product structure (Section 6.4) 21
14.5 Execution output: INSERT commands (Section 7.1) 21
14.6 Execution output: UPDATE and DELETE commands (Sections 7.2 and 7.3) 21
14.7 Execution output: Basic SELECT queries (Section 8.1) 22
14.8 Execution output: JOIN operations: catalogue and orders with customers (Section 8.2) 22
14.9 Execution output: JOIN operations: order items and payment/shipping overview (Section 8.2) . 22
14.10 Execution output: Aggregate functions (Section 8.3) 23
14.11 Execution output: GROUP BY and HAVING (Section 8.4) 23
14.12 Execution output: Subqueries (Section 8.5) 23
14.13 Execution output: Views (Section 8.6) 24
14.14 Execution output: Transaction with SAVEPOINT and COMMIT (Section 9.2) 24
14.15 Execution output: Row locking with SELECT ... FOR UPDATE (Section 9.3) 24
14.16 Execution output: Stored function Customer_Order_Count (Section 10.1) 25
14.17 Execution output: Stored procedure Update_Order_Status (Section 10.2) 25
14.18 Execution output: Cursor procedure List_Low_Stock (Section 10.3) 25
14.19 Execution output: Trigger Validate_Order_Item (Section 10.4) 26
14.20 Execution output: Index creation (Section 11.2) 26
14.21 Execution output: Test cases TC01–TC06 (Section 12), run inside a transaction and rolled back 26
14.22 Execution output: Test cases TC07–TC12 (Section 12), run inside a transaction and rolled back 27
14.23 Execution output: Order report and order item details (Sections 13.1 and 13.2) 27
14.24 Execution output: Cancellation, revenue, inventory and shipping reports (Sections 13.3–13.6) . 27

Abstract

The **E-Commerce Order Flow** project is designed to organize the major activities involved in online shopping using a centralized relational database. The system maintains customer information, product details, product categories, orders, individual order items, payments, and shipping information. It supports the movement of an order from product selection and order placement to payment confirmation, dispatch, and delivery tracking.

The database is designed around related tables connected through primary and foreign keys. Constraints help prevent invalid values and maintain consistency between orders, products, payments, and shipments. Normalization is applied to reduce unnecessary repetition, while SQL queries, joins, aggregate functions, views, and transactions support data retrieval and order analysis. MySQL is used as the database platform.

The report explains requirement analysis, ER modelling, relational schema design, normalization, SQL DDL and DML operations, queries, transaction handling, stored programs, indexing, testing, and report generation. The project demonstrates how DBMS concepts can support reliable order management and provide a foundation for future enhancements such as customer authentication, automated notifications, and real-time shipment tracking.

Keywords: E-commerce, order flow, customer management, product catalogue, inventory, order processing, payment, shipping, MySQL, SQL, normalization, data integrity.

CHAPTER 1

Introduction

Online shopping has become an important part of modern business. An e-commerce platform must coordinate several connected activities: maintaining a product catalogue, registering customers, recording orders, checking stock, collecting payments, and arranging delivery. Each activity produces data that must remain consistent with the others. If these records are kept in separate files or updated manually, duplicate information, incorrect totals, stock errors, and delayed order updates may occur.

The **E-Commerce Order Flow** project uses a relational database to store and manage this information in a structured form. Customers are associated with their orders, each order contains one or more order items, products belong to categories, payments are associated with orders, and shipping records store delivery information. Primary keys identify records uniquely, while foreign keys connect related records.

The project is a DBMS laboratory implementation rather than a complete commercial shopping website. Its main focus is the design and implementation of the database layer using MySQL and SQL. Sample data and queries demonstrate how common order-management tasks can be performed.

1.1 Background

A typical online purchase starts when a customer browses a product and selects a quantity. The system records an order and the products included in it. The order total is calculated from item quantities and unit prices. Payment information is then stored, and a shipping record is used to follow dispatch and delivery. Product stock must be monitored so that the catalogue does not show incorrect availability.

These operations are related. For example, an order item must refer to a valid product, and a payment must refer to a valid order. A relational database provides a systematic way to represent these dependencies and apply constraints.

1.2 Need for the System

The system is needed to:

- maintain customer, product, category, order, payment, and shipping records in one place;
- reduce duplication and inconsistent values;
- retrieve order histories and product details quickly;
- track payment and delivery statuses;
- support inventory and sales summaries through SQL queries; and
- preserve relationships using keys and integrity constraints.

1.3 Scope

The scope includes database design, sample record insertion, order and payment queries, inventory checks, transaction examples, indexing, and testing scenarios. A graphical user interface, live payment gateway, courier API, and production authentication are outside the current implementation and are considered possible extensions.

1.4 Organization of the Report

The report begins with problem analysis and requirements, followed by ER modelling and the relational model. It then discusses normalization, SQL schema creation, data manipulation, queries and views, transactions, stored programs, indexing, testing, reports, results, SDG mapping, future scope, and bibliography.

CHAPTER 2

Problem Analysis

The main challenge in order processing is maintaining accurate information while multiple related operations occur. A customer may place several orders, an order may contain several products, and a product may appear in many orders. Payments and shipping updates must be linked to the correct order. If the same customer or product details are repeatedly stored in order records, an update to those details may need to be repeated in many places.

2.1 Problem Statement

Manual or poorly structured order records can lead to duplicate entries, incorrect order totals, inaccurate stock values, difficulty finding an order's payment status, and incomplete delivery information. The project proposes a centralized relational database that organizes the data into connected tables and uses SQL constraints and queries to support accurate e-commerce order processing.

2.2 Users and Assumptions

The main users and their responsibilities are summarized in Table 2.1.

Table 2.1: Users of the E-Commerce Order Flow System

User	Responsibilities
Customer	Browses products, provides contact details, places orders, and checks order progress.
Administrator	Maintains products, categories, prices, and stock records.
Order operator	Reviews orders and updates order status.
Payment operator	Records payment method, amount, and payment status.
Shipping operator	Records tracking details and updates dispatch and delivery status.
Manager	Reviews sales, pending orders, payment summaries, and inventory reports.

The design uses these assumptions:

1. Every customer is identified by a unique customer ID.
2. Each product has a unique product ID and belongs to one category.
3. An order belongs to one customer and contains one or more order items.
4. Each order item refers to one product and stores the quantity and unit price used for that order.
5. An order may have a payment record and a shipping record; the schema can be extended to support multiple payments or shipments if required.
6. Stock quantity cannot be negative.
7. Order, payment, and shipping statuses must use values defined by the database design.

2.3 Functional Requirements

The system should create and maintain customer records, categories, products, orders, order items, payments, and shipping details. It should retrieve available products, calculate order-item subtotals, list the items in an order, show payment status, and track shipping status. It should also support sales totals, order counts, stock summaries, and lists of pending or delivered orders.

2.4 Non-Functional Requirements

Integrity Key constraints and validation rules must prevent invalid references.

Maintainability Related data should be separated into logical tables.

Usability SQL queries should provide understandable order and sales information.

Performance Indexes should be considered for frequently searched fields.

Reliability Transactions should be used when several changes form one logical operation.

2.5 Limitations

The project uses sample records and a database-level implementation. Real-world deployment would require security controls, access roles, privacy protections, application validation, payment-provider integration, and operational monitoring.

CHAPTER 3

ER Model

The Entity–Relationship (ER) model describes the main objects in the E-Commerce Order Flow system and the relationships between them. Entities are converted into relational tables during implementation. Each entity has attributes, and a primary key uniquely identifies each record.

3.1 Entities and Attributes

Table 3.1: Entities and Principal Attributes

Entity	Attributes	Key
Customer	Customer_ID, Name, Email, Phone, Address, Registration_Date	PK: Customer_ID
Category	Category_ID, Category_Name, Description	PK: Category_ID
Product	Product_ID, Product_Name, Description, Price, Stock_Quantity, Category_ID	PK: Product_ID; FK: Category_ID
Orders	Order_ID, Customer_ID, Order_Date, Status, Total_Amount	PK: Order_ID; FK: Customer_ID
Order Item	Order_Item_ID, Order_ID, Product_ID, Quantity, Unit_Price, Subtotal	PK: Order_Item_ID; FKs: Order_ID, Product_ID
Payment	Payment_ID, Order_ID, Payment_Date, Amount, Payment_Method, Payment_Status, Transaction_ID	PK: Payment_ID; FK: Order_ID
Shipping	Shipping_ID, Order_ID, Delivery_Address, Shipping_Date, Delivery_Status, Tracking_ID, Carrier_Name	PK: Shipping_ID; FK: Order_ID

3.2 Relationships

A customer can place many orders, while each order belongs to one customer. A category can contain many products, while each product belongs to one category. An order contains one or more order items, and each order item references one product. A product may therefore appear in many order items over time. Payment and shipping information are associated with an order.

3.3 Cardinality and Participation

Table 3.2: Relationships, Cardinality, and Participation

Relationship	Entities	Cardinality	Explanation
Places	Customer – Orders	1:N	A customer may place multiple orders.
Contains	Orders – Order Item	1:N	An order can contain multiple line items.
Appears in	Product – Order Item	1:N	A product may be included in many order items.
Belongs to	Category – Product	1:N	A category may classify multiple products.
Has payment	Orders – Payment	1:1	One payment record is associated with an order in the simplified design.
Has shipping	Orders – Shipping	1:1	One shipping record is associated with an order in the simplified design.

3.4 ER Diagram

Figure 3.1 provides a conceptual view of the main entities. It is a simplified representation; the relational schema and foreign-key constraints provide the exact implementation details.

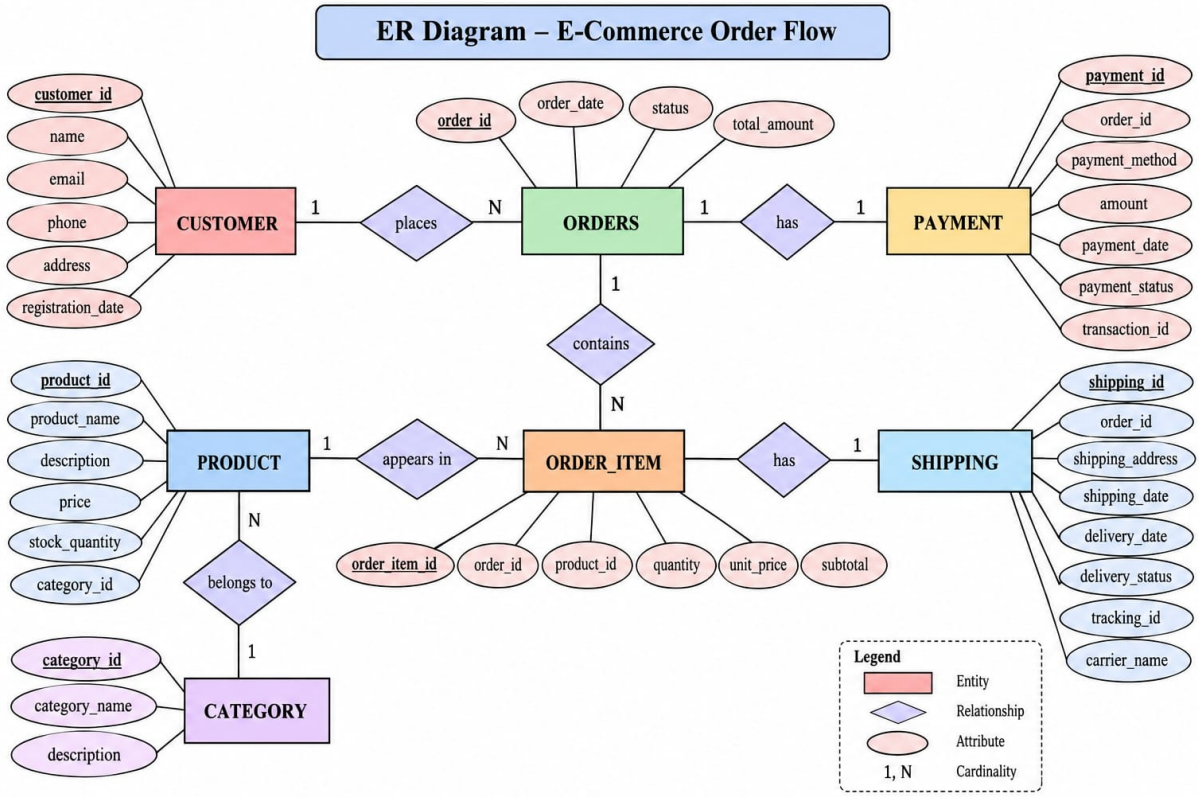


Figure 3.1: ER Diagram of the E-Commerce Order Flow System

CHAPTER 4

Relational Model

The relational model represents data as tables. A row represents one record, and a column represents an attribute. The primary key identifies each row, while foreign keys maintain valid references between related tables.

4.1 Relational Schema

The proposed schema is:

- **Customer**(Customer_ID, Name, Email, Phone, Address, Registration_Date)
- **Category**(Category_ID, Category_Name, Description)
- **Product**(Product_ID, Product_Name, Description, Price, Stock_Quantity, Category_ID)
- **Orders**(Order_ID, Customer_ID, Order_Date, Status, Total_Amount)
- **Order_Item**(Order_Item_ID, Order_ID, Product_ID, Quantity, Unit_Price, Subtotal)
- **Payment**(Payment_ID, Order_ID, Payment_Date, Amount, Payment_Method, Payment_Status, Transaction_ID)
- **Shipping**(Shipping_ID, Order_ID, Delivery_Address, Shipping_Date, Delivery_Status, Tracking_ID, Carrier_Name)

4.2 Primary Keys and Foreign Keys

Customer_ID, Category_ID, Product_ID, Order_ID, Order_Item_ID, Payment_ID, and Shipping_ID are primary keys. Product.Category_ID references Category.Category_ID. Orders.Customer_ID references Customer.Customer_ID. Order_Item.Order_ID and Order_Item.Product_ID reference Orders and Product. Payment.Order_ID and Shipping.Order_ID reference Orders.

Table 4.1: Foreign-Key Relationships

Table	Foreign Key	References
Product	Category_ID	Category(Category_ID)
Orders	Customer_ID	Customer(Customer_ID)
Order_Item	Order_ID	Orders(Order_ID)
Order_Item	Product_ID	Product(Product_ID)
Payment	Order_ID	Orders(Order_ID)
Shipping	Order_ID	Orders(Order_ID)

4.3 Integrity Rules

The design uses NOT NULL for required information, UNIQUE for fields such as customer email and tracking ID where appropriate, CHECK constraints for positive price and quantity values, and foreign keys to stop records from referring to missing parents. Status fields are restricted to known values. In MySQL, CHECK constraints are enforced in modern versions, including MySQL 8.0.16 and later.

CHAPTER 5

Normalization

Normalization organizes data to reduce redundancy and prevent insertion, update, and deletion anomalies. The design is intended to reach Third Normal Form (3NF) by storing customer, category, product, order, order-item, payment, and shipping information separately.

5.1 Functional Dependencies

Representative functional dependencies are:

- Customer_ID → Name, Email, Phone, Address, Registration_Date
- Category_ID → Category_Name, Description
- Product_ID → Product_Name, Description, Price, Stock_Quantity, Category_ID
- Order_ID → Customer_ID, Order_Date, Status, Total_Amount
- Order_Item_ID → Order_ID, Product_ID, Quantity, Unit_Price, Subtotal
- Payment_ID → Order_ID, Payment_Date, Amount, Payment_Method, Payment_Status
- Shipping_ID → Order_ID, Delivery_Address, Shipping_Date, Delivery_Status

5.2 First Normal Form (1NF)

A relation is in 1NF when each field contains a single atomic value and there are no repeating groups. A poorly designed order sheet may store several products in a single cell, for example "Mouse x 1, Keyboard x 2". This makes it difficult to query one product or update one quantity.

Table 5.1: Illustration of a Non-Atomic Order Record

Order ID	Customer	Products in One Field
5001	Asha Rao	Mouse (1), Keyboard (2), USB Hub (1)

To achieve 1NF, each product is stored as a separate order-item row. Each row contains one order ID, one product ID, one quantity, and one unit price.

Table 5.2: Order Items in 1NF

Order ID	Product ID	Quantity	Unit Price
5001	101	1	650.00
5001	102	2	900.00
5001	103	1	400.00

5.3 Second Normal Form (2NF)

A relation is in 2NF when it is in 1NF and each non-key attribute depends on the whole candidate key. If a combined table uses (Order_ID, Product_ID) as a key, customer name depends only on Order_ID and product name depends only on Product_ID. These are partial dependencies. Separating Orders, Product, and Order_Item removes the partial dependencies.

5.4 Third Normal Form (3NF)

A relation is in 3NF when it is in 2NF and non-key attributes do not depend transitively on a key. For example, Category_Name depends on Category_ID, not directly on Product_ID. Therefore, category information is stored in Category, while Product keeps only Category_ID as a foreign key. Similarly, customer details are stored in Customer and Orders keeps Customer_ID.

Table 5.3: Final Normalized Relations

No.	Relation	Primary Key	Foreign Keys
1	Customer	Customer_ID	–
2	Category	Category_ID	–
3	Product	Product_ID	Category_ID
4	Orders	Order_ID	Customer_ID
5	Order_Item	Order_Item_ID	Order_ID, Product_ID
6	Payment	Payment_ID	Order_ID
7	Shipping	Shipping_ID	Order_ID

Normalization makes updates easier and reduces repeated customer and product details. It does not remove the need to validate derived totals and stock changes at the application or transaction level.

CHAPTER 6

SQL DDL Commands

Data Definition Language (DDL) commands create and modify database objects. The following examples use MySQL syntax. Run the table definitions in dependency order so referenced tables exist before child tables.

6.1 Database Creation

```
CREATE DATABASE ecommerce_order_flow;
USE ecommerce_order_flow;
```

6.2 Table Creation

```
CREATE TABLE Customer (
  Customer_ID       INT PRIMARY KEY,
  Name              VARCHAR(100) NOT NULL,
  Email             VARCHAR(120) NOT NULL UNIQUE,
  Phone             VARCHAR(20),
  Address           VARCHAR(255),
  Registration_Date DATE DEFAULT (CURRENT_DATE)
);

CREATE TABLE Category (
  Category_ID   INT PRIMARY KEY,
  Category_Name VARCHAR(80) NOT NULL UNIQUE,
  Description   VARCHAR(255)
);

CREATE TABLE Product (
  Product_ID     INT PRIMARY KEY,
  Product_Name   VARCHAR(120) NOT NULL,
  Description    VARCHAR(255),
  Price          DECIMAL(10,2) NOT NULL CHECK (Price > 0),
  Stock_Quantity INT NOT NULL DEFAULT 0 CHECK (Stock_Quantity >= 0),
  Category_ID    INT NOT NULL,
  FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID)
);

CREATE TABLE Orders (
  Order_ID     INT PRIMARY KEY,
  Customer_ID  INT NOT NULL,
  Order_Date   DATETIME DEFAULT CURRENT_TIMESTAMP,
  Status       VARCHAR(20) NOT NULL DEFAULT 'Placed',
  Total_Amount DECIMAL(10,2) NOT NULL DEFAULT 0 CHECK (Total_Amount >= 0),
  CHECK (Status IN
    ('Placed','Confirmed','Packed','Shipped','Delivered','Cancelled')),
  FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
);

CREATE TABLE Order_Item (
  Order_Item_ID INT PRIMARY KEY,
  Order_ID      INT NOT NULL,
  Product_ID    INT NOT NULL,
  Quantity      INT NOT NULL CHECK (Quantity > 0),
  Unit_Price    DECIMAL(10,2) NOT NULL CHECK (Unit_Price > 0),
  Subtotal      DECIMAL(10,2) NOT NULL CHECK (Subtotal >= 0),
  FOREIGN KEY (Order_ID)   REFERENCES Orders(Order_ID),
  FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)
);

CREATE TABLE Payment (
  Payment_ID     INT PRIMARY KEY,
  Order_ID       INT NOT NULL UNIQUE,
  Payment_Date   DATETIME DEFAULT CURRENT_TIMESTAMP,
  Amount         DECIMAL(10,2) NOT NULL CHECK (Amount >= 0),
  Payment_Method VARCHAR(20) NOT NULL,
  Payment_Status VARCHAR(20) NOT NULL DEFAULT 'Pending',
  Transaction_ID VARCHAR(100) UNIQUE,
  CHECK (Payment_Method IN ('UPI','Card','Cash','Net Banking')),
  CHECK (Payment_Status IN ('Pending','Paid','Failed','Refunded')),
  FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
);

CREATE TABLE Shipping (
  Shipping_ID      INT PRIMARY KEY,
  Order_ID         INT NOT NULL UNIQUE,
  Delivery_Address VARCHAR(255) NOT NULL,
  Shipping_Date    DATE,
  Delivery_Status  VARCHAR(20) NOT NULL DEFAULT 'Processing',
  Tracking_ID      VARCHAR(100) UNIQUE,
  Carrier_Name     VARCHAR(80),
  CHECK (Delivery_Status IN
    ('Processing','Packed','Shipped','Out for Delivery','Delivered','Returned')),
```

```
  FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
);
```

6.3 Constraints

Primary keys prevent duplicate identifiers. Foreign keys enforce valid relationships. NOT NULL ensures required fields are supplied, UNIQUE prevents repeated values in selected fields, and CHECK rules reject invalid prices, quantities, amounts, and status values. DEFAULT provides initial values when an insert does not supply one.

6.4 ALTER Operation

ALTER modifies an existing table. For example, a product can be given an additional brand field:

```
ALTER TABLE Product ADD Brand VARCHAR(80);
ALTER TABLE Customer MODIFY Phone VARCHAR(25);
```

6.5 DROP Operation

DROP removes a database object and its data. It should be used carefully, particularly when other tables reference the object.

```
-- Example only; do not run unless removal is intended.
DROP TABLE Shipping;
```

CHAPTER 7

SQL DML Commands

Data Manipulation Language commands add, modify, and remove records. The examples below use sample values to demonstrate the order flow.

7.1 INSERT

```
INSERT INTO Customer (Customer_ID, Name, Email, Phone, Address)
VALUES
  (1, 'Asha Rao',  'asha@example.com',  '9876500001', 'Hyderabad'),
  (2, 'Rahul Das', 'rahul@example.com', '9876500002', 'Secunderabad');

INSERT INTO Category VALUES
  (10, 'Electronics', 'Electronic accessories and devices'),
  (20, 'Home',        'Home and kitchen products'),
  (30, 'Books',       'Printed learning materials');

INSERT INTO Product
  (Product_ID, Product_Name, Description, Price, Stock_Quantity, Category_ID)
VALUES
  (101, 'Wireless Mouse', 'USB wireless mouse',         650.00, 40, 10),
  (102, 'Keyboard',       'Standard USB keyboard',      900.00, 25, 10),
  (103, 'Water Bottle',   'Reusable steel bottle',      400.00, 50, 20),
  (104, 'SQL Basics',     'Introductory database book', 550.00, 30, 30);

INSERT INTO Orders (Order_ID, Customer_ID, Status, Total_Amount)
VALUES
  (5001, 1, 'Placed',    1550.00),
  (5002, 2, 'Confirmed', 950.00);

INSERT INTO Order_Item VALUES
  (9001, 5001, 101, 1, 650.00, 650.00),
  (9002, 5001, 102, 1, 900.00, 900.00),
  (9003, 5002, 103, 1, 400.00, 400.00),
  (9004, 5002, 104, 1, 550.00, 550.00);

INSERT INTO Payment
  (Payment_ID, Order_ID, Amount, Payment_Method, Payment_Status, Transaction_ID)
VALUES
  (7001, 5001, 1550.00, 'UPI',  'Paid',    'TXN-5001'),
  (7002, 5002, 950.00,  'Card', 'Pending', 'TXN-5002');

INSERT INTO Shipping
  (Shipping_ID, Order_ID, Delivery_Address, Delivery_Status, Tracking_ID, Carrier_Name)
VALUES
  (8001, 5001, 'Hyderabad',    'Shipped',    'TRK-5001', 'Demo Express'),
  (8002, 5002, 'Secunderabad', 'Processing', 'TRK-5002', 'Demo Express');
```

Note: The sample order totals are illustrative. In an application, the order total should be calculated from the associated order items rather than trusted from user input.

7.2 UPDATE

UPDATE changes existing records. An order status can be updated as the order moves through processing.

```
UPDATE Orders  SET Status = 'Packed'         WHERE Order_ID   = 5002;
UPDATE Product SET Price = 680.00            WHERE Product_ID = 101;
UPDATE Payment SET Payment_Status = 'Paid'   WHERE Payment_ID = 7002;
```

7.3 DELETE

DELETE removes selected records. Child records may need to be removed first, or the database may reject the operation because of foreign-key restrictions.

```
-- Example only: remove a shipping record for a test order.
DELETE FROM Shipping
WHERE Shipping_ID = 8002;
```

Deleting operational records is generally less suitable than marking them cancelled or returned, because retained records support auditing and reporting.

CHAPTER 8

SQL Queries

SQL queries retrieve and analyze data. The project uses basic SELECT statements, joins, aggregate functions, grouping, subqueries, and views.

8.1 Basic SELECT Queries

```sql
-- Display all products.
SELECT * FROM Product;

-- Display products that are in stock.
SELECT Product_ID, Product_Name, Price
FROM   Product
WHERE  Stock_Quantity > 0;

-- Display products in ascending price order.
SELECT Product_Name, Price
FROM   Product
ORDER BY Price ASC;

-- Find a customer by email.
SELECT Customer_ID, Name, Phone
FROM   Customer
WHERE  Email = 'asha@example.com';
```

8.2 JOIN Operations

JOIN combines related rows from different tables.

```sql
-- Product catalogue with category names.
SELECT P.Product_Name, P.Price, P.Stock_Quantity, C.Category_Name
FROM   Product P
JOIN   Category C ON P.Category_ID = C.Category_ID;

-- Orders with customer information.
SELECT O.Order_ID, C.Name, O.Order_Date, O.Status, O.Total_Amount
FROM   Orders O
JOIN   Customer C ON O.Customer_ID = C.Customer_ID
ORDER BY O.Order_Date DESC;

-- Detailed list of items within each order.
SELECT O.Order_ID, C.Name, P.Product_Name,
       I.Quantity, I.Unit_Price, I.Subtotal
FROM   Orders O
JOIN   Customer   C ON O.Customer_ID = C.Customer_ID
JOIN   Order_Item I ON O.Order_ID    = I.Order_ID
JOIN   Product    P ON I.Product_ID  = P.Product_ID
ORDER BY O.Order_ID, I.Order_Item_ID;

-- Order payment and shipping overview.
SELECT O.Order_ID, O.Status, Pay.Payment_Status,
       S.Delivery_Status, S.Tracking_ID
FROM   Orders O
LEFT JOIN Payment Pay ON O.Order_ID = Pay.Order_ID
LEFT JOIN Shipping S  ON O.Order_ID = S.Order_ID;
```

8.3 Aggregate Functions

COUNT, SUM, AVG, MIN, and MAX summarize data.

```sql
SELECT COUNT(*)    AS Total_Products        FROM Product;
SELECT COUNT(*)    AS Total_Orders          FROM Orders;
SELECT SUM(Amount) AS Paid_Revenue          FROM Payment
WHERE  Payment_Status = 'Paid';
SELECT AVG(Price)  AS Average_Product_Price FROM Product;
SELECT MAX(Price)  AS Highest_Price,
       MIN(Price)  AS Lowest_Price          FROM Product;
```

8.4 GROUP BY and HAVING

GROUP BY forms groups of records, while HAVING filters those groups.

```sql
-- Number of products in each category.
SELECT C.Category_Name, COUNT(P.Product_ID) AS Product_Count
FROM   Category C
LEFT JOIN Product P ON C.Category_ID = P.Category_ID
GROUP BY C.Category_ID, C.Category_Name;
```

```sql
-- Number of orders in each status.
SELECT Status, COUNT(*) AS Order_Count
FROM   Orders
GROUP BY Status;

-- Customers with more than one order.
SELECT Customer_ID, COUNT(*) AS Number_Of_Orders
FROM   Orders
GROUP BY Customer_ID
HAVING COUNT(*) > 1;
```

8.5 Subqueries

A subquery is a query nested inside another query.

```sql
-- Products priced above the average price.
SELECT Product_Name, Price
FROM   Product
WHERE  Price > (SELECT AVG(Price) FROM Product);

-- Customers who have placed at least one order.
SELECT Customer_ID, Name
FROM   Customer
WHERE  Customer_ID IN (SELECT Customer_ID FROM Orders);

-- Products that have never appeared in an order item.
SELECT Product_ID, Product_Name
FROM   Product
WHERE  Product_ID NOT IN (SELECT Product_ID FROM Order_Item);
```

8.6 Views

A view is a virtual table based on a query. It can simplify frequently used joins.

```sql
CREATE VIEW Order_Summary AS
SELECT O.Order_ID, C.Name AS Customer_Name,
       O.Order_Date, O.Status, O.Total_Amount
FROM   Orders O
JOIN   Customer C ON O.Customer_ID = C.Customer_ID;

SELECT * FROM Order_Summary;

CREATE VIEW Product_Stock_View AS
SELECT Product_ID, Product_Name, Price, Stock_Quantity
FROM   Product
WHERE  Stock_Quantity > 0;
```

Views provide a reusable presentation of data, but permissions should be applied before exposing customer or payment information to users.

CHAPTER 9

Transactions and Concurrency

A transaction is a group of database operations treated as one logical unit. Transactions are important when an order operation includes more than one change, such as inserting an order, inserting its items, updating stock, and recording payment information.

9.1 Transaction Management

The ACID properties describe reliable transaction behavior:

- **Atomicity:** all operations in a transaction succeed together or are rolled back.
- **Consistency:** constraints remain satisfied before and after the transaction.
- **Isolation:** concurrent transactions should not interfere in a way that corrupts results.
- **Durability:** committed changes remain stored.

9.2 COMMIT, ROLLBACK, and SAVEPOINT

COMMIT permanently saves changes. ROLLBACK cancels uncommitted changes. SAVEPOINT marks a point inside a transaction to which a rollback can return.

```
START TRANSACTION;

UPDATE Product
SET    Stock_Quantity = Stock_Quantity - 1
WHERE  Product_ID = 101 AND Stock_Quantity >= 1;

-- Application should verify that exactly one row was updated.
INSERT INTO Orders (Order_ID, Customer_ID, Status, Total_Amount)
VALUES (5010, 1, 'Placed', 650.00);

SAVEPOINT after_order;

INSERT INTO Order_Item
VALUES (9010, 5010, 101, 1, 650.00, 650.00);

-- If all checks succeed:
COMMIT;

-- If validation fails before COMMIT, use:
-- ROLLBACK TO after_order;
-- or ROLLBACK;
```

The example illustrates transaction structure. A production application should check stock and affected-row counts and should roll back if any required step fails.

9.3 Concurrency Control

Concurrency control is needed when two customers attempt to purchase the last unit of a product at nearly the same time. A transaction can lock the product row while checking and changing its stock.

```
START TRANSACTION;

SELECT Stock_Quantity
FROM   Product
WHERE  Product_ID = 101
FOR UPDATE;

-- Application checks that stock is greater than zero.
UPDATE Product
SET    Stock_Quantity = Stock_Quantity - 1
WHERE  Product_ID = 101 AND Stock_Quantity > 0;

COMMIT;
```

Row locking and conditional updates reduce the risk of overselling. The application should verify the update count and cancel the transaction if stock was unavailable.

9.4 Order Cancellation

A cancellation should update the order status and, if business rules permit, restore reserved stock. The stock restoration must be performed only once; repeated cancellation requests should not increase stock repeatedly. This requires a transaction and validation of the order's current status.

CHAPTER 10

Stored Programs and Triggers

MySQL supports stored procedures, functions, cursors, and triggers. These database objects can encapsulate reusable logic. The following examples are illustrative MySQL routines; delimiter commands are entered in a MySQL client and are not SQL statements sent unchanged by every application connector.

10.1 Stored Function

The function below returns the number of orders placed by a customer.

```
DELIMITER //

CREATE FUNCTION Customer_Order_Count(p_customer_id INT)
RETURNS INT
READS SQL DATA
BEGIN
  DECLARE v_count INT DEFAULT 0;

  SELECT COUNT(*) INTO v_count
  FROM   Orders
  WHERE  Customer_ID = p_customer_id;

  RETURN v_count;
END //

DELIMITER ;

SELECT Customer_Order_Count(1);
```

10.2 Stored Procedure

This procedure changes an order status. In a full application, allowed transitions should be validated against the current status and user permissions.

```
DELIMITER //

CREATE PROCEDURE Update_Order_Status(
  IN p_order_id   INT,
  IN p_new_status VARCHAR(20)
)
BEGIN
  IF p_new_status NOT IN
      ('Placed','Confirmed','Packed','Shipped','Delivered','Cancelled') THEN
    SIGNAL SQLSTATE '45000'
       SET MESSAGE_TEXT = 'Invalid order status';
  END IF;

  UPDATE Orders
  SET    Status = p_new_status
  WHERE  Order_ID = p_order_id;

  IF ROW_COUNT() = 0 THEN
    SIGNAL SQLSTATE '45000'
       SET MESSAGE_TEXT = 'Order not found';
  END IF;
END //

DELIMITER ;

CALL Update_Order_Status(5001, 'Delivered');
```

10.3 Cursor

A cursor processes query results row by row inside a stored program. Cursors are useful for procedural tasks, although set-based SQL is usually preferred for simple bulk operations.

```
DELIMITER //

CREATE PROCEDURE List_Low_Stock()
BEGIN
  DECLARE finished INT DEFAULT 0;
  DECLARE v_id     INT;
  DECLARE v_name   VARCHAR(120);
  DECLARE v_stock  INT;

  DECLARE stock_cursor CURSOR FOR
    SELECT Product_ID, Product_Name, Stock_Quantity
    FROM   Product
    WHERE  Stock_Quantity < 10;
```

```
  DECLARE CONTINUE HANDLER FOR NOT FOUND
    SET finished = 1;

  OPEN stock_cursor;

  read_loop: LOOP
    FETCH stock_cursor INTO v_id, v_name, v_stock;
    IF finished = 1 THEN
      LEAVE read_loop;
    END IF;
    SELECT v_id    AS Product_ID,
           v_name  AS Product_Name,
           v_stock AS Stock_Quantity;
  END LOOP;

  CLOSE stock_cursor;
END //

DELIMITER ;
```

10.4 Trigger

A trigger can reject an order item with an invalid quantity. The CHECK constraint on the table already enforces this rule in modern MySQL; the trigger is shown to demonstrate trigger syntax.

```
DELIMITER //

CREATE TRIGGER Validate_Order_Item
BEFORE INSERT ON Order_Item
FOR EACH ROW
BEGIN
  IF NEW.Quantity <= 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Quantity must be positive';
  END IF;

  IF NEW.Unit_Price <= 0 THEN
    SIGNAL SQLSTATE '45000'
       SET MESSAGE_TEXT = 'Unit price must be positive';
  END IF;
END //

DELIMITER ;
```

Triggers should be kept focused and documented. Complex order totals, inventory reservation, and cancellation logic should be designed carefully to avoid hidden side effects or repeated updates.

CHAPTER 11

File Organization and Indexing

File organization describes how database records are stored, while indexing provides a faster way to locate records. Indexes are useful for fields that frequently appear in search conditions, joins, or sorting operations. They also consume storage and can slow inserts and updates, so they should be selected based on expected queries.

11.1 File Organization

Common approaches include heap organization, where records are stored without a specific order; sequential organization, where records are arranged by a key; and indexed organization, where an index helps locate records. MySQL's InnoDB engine organizes table data around a clustered primary key and uses secondary indexes for other search fields.

11.2 Indexing in the Project

Customer email, product category, order date, order status, and foreign-key fields may be used frequently in queries. Primary-key and UNIQUE constraints create indexes automatically. Additional indexes should be added only when they support actual query patterns.

```
CREATE INDEX idx_product_category     ON Product(Category_ID);
CREATE INDEX idx_orders_customer_date ON Orders(Customer_ID, Order_Date);
CREATE INDEX idx_orders_status        ON Orders(Status);
CREATE INDEX idx_order_item_product   ON Order_Item(Product_ID);
```

11.3 B+ Tree Indexing

B+ trees are balanced tree structures widely used in database indexes. Internal nodes guide the search, and leaf nodes contain keys with references to records. Linked leaf nodes support efficient range scans. An index on (Customer_ID, Order_Date) can help retrieve a customer's orders in date order.

11.4 Index Selection

Indexes should be evaluated using representative data and query plans. Too many indexes increase storage use and add overhead to INSERT, UPDATE, and DELETE operations. The best design balances read performance with the cost of maintaining indexes.

CHAPTER 12

Testing

Testing checks whether the database accepts valid information, rejects invalid information, and returns the expected results. The test cases below describe the expected behavior of the proposed schema and queries. Mark a test as passed only after executing it in the configured MySQL database.

12.1 Test Cases

Table 12.1: Test Cases for E-Commerce Order Flow

Test ID	Test	Expected Result	Status
TC01	Insert a valid customer	Customer row is inserted	To verify
TC02	Insert duplicate customer email	UNIQUE constraint rejects the duplicate	To verify
TC03	Insert a product with negative price	CHECK constraint rejects the row	To verify
TC04	Insert product with missing category ID	Foreign-key constraint rejects the row	To verify
TC05	Add a valid order item	Item is linked to an existing order and product	To verify
TC06	Insert quantity zero	CHECK constraint or trigger rejects it	To verify
TC07	Retrieve an order with JOIN	Customer and item details are returned	To verify
TC08	Update order status to an allowed value	Status is updated	To verify
TC09	Update order status to an invalid value	Validation rejects the value	To verify
TC10	Query products with zero stock	No unavailable product is listed as in stock	To verify
TC11	Record payment for a valid order	Payment references an existing order	To verify
TC12	Run sales aggregation	Query returns totals for matching paid payments	To verify

12.2 Normal and Boundary Cases

Normal cases use valid customer, product, order, and payment values. Boundary cases include a quantity of one, stock reaching zero, a product at the lowest allowed positive price, and an order with a zero total if the business rules permit it. Each boundary should be checked against the rules actually defined in the schema.

12.3 Invalid Input Testing

Invalid input cases include duplicate emails, missing customer IDs, negative prices, negative stock, invalid status values, and order items referencing nonexistent products. The database should reject such records through keys and constraints rather than silently accepting inconsistent data.

12.4 Transaction Testing

Transaction testing should verify that partial operations are not left behind when one step fails. For example, if an order is inserted but stock cannot be reduced, the transaction should be rolled back. Concurrent purchase tests should confirm that two transactions cannot both successfully decrement the same final unit.

12.5 Test Evidence

For final submission, capture actual MySQL Workbench or command-line output for table creation, SELECT queries, and test cases. The report should not claim execution results that have not been observed. The expected results in Table 12.1 provide a checklist for collecting this evidence.

CHAPTER 13

Reports

Reports convert the stored records into information that supports daily operations and decision-making. The queries below can be used to generate order, cancellation, revenue, inventory, and shipping summaries.

13.1 Order Report

The order report lists the order ID, customer, date, status, and total.

```
SELECT O.Order_ID, C.Name AS Customer_Name,
       O.Order_Date, O.Status, O.Total_Amount
FROM   Orders O
JOIN   Customer C ON O.Customer_ID = C.Customer_ID
ORDER BY O.Order_Date DESC;
```

13.2 Order Item Details

This report lists the products and quantities associated with each order.

```
SELECT I.Order_ID, P.Product_Name, I.Quantity, I.Unit_Price, I.Subtotal
FROM   Order_Item I
JOIN   Product P ON I.Product_ID = P.Product_ID
ORDER BY I.Order_ID;
```

13.3 Cancellation Report

The cancellation report identifies orders that have been cancelled.

```
SELECT O.Order_ID, C.Name, O.Order_Date, O.Status
FROM   Orders O
JOIN   Customer C ON O.Customer_ID = C.Customer_ID
WHERE  O.Status = 'Cancelled'
ORDER BY O.Order_Date DESC;
```

13.4 Revenue Report

The revenue report totals payments marked as paid. It reports recorded payments rather than assuming that every placed order has been paid.

```
SELECT Payment_Method, COUNT(*) AS Payment_Count, SUM(Amount) AS Total_Revenue
FROM   Payment
WHERE  Payment_Status = 'Paid'
GROUP BY Payment_Method;
```

13.5 Inventory Report

The inventory report shows current stock and identifies products that need replenishment.

```
SELECT Product_ID, Product_Name, Stock_Quantity
FROM   Product
WHERE  Stock_Quantity < 10
ORDER BY Stock_Quantity ASC;
```

13.6 Shipping Report

The shipping report helps operators find orders that are still being processed or are in transit.

```
SELECT O.Order_ID, C.Name, S.Carrier_Name, S.Tracking_ID, S.Delivery_Status
FROM   Shipping S
JOIN   Orders   O ON S.Order_ID    = O.Order_ID
JOIN   Customer C ON O.Customer_ID = C.Customer_ID
WHERE  S.Delivery_Status <> 'Delivered'
ORDER BY O.Order_Date;
```

These reports can be exported or used as the basis for dashboards. In a real system, access to personal information should be restricted to authorized users.

CHAPTER 14

Execution Outputs

The following screenshots show the actual output of the SQL statements in this report, executed in the MySQL 8.0.46 command-line client on database `ecommerce_order_flow`, in the order they appear in Chapters 6–13.

```
mysql  Ver 8.0.46  -  root@localhost : ecommerce_order_flow

mysql> CREATE DATABASE ecommerce_order_flow;
Query OK, 1 row affected (0.00 sec)

mysql> USE ecommerce_order_flow;
Database changed
```

Figure 14.1: Execution output: Database creation (Section 6.1)

```
mysql  Ver 8.0.46  -  root@localhost : ecommerce_order_flow

mysql> CREATE TABLE Customer (
    ->   Customer_ID       INT PRIMARY KEY,
    ->   Name              VARCHAR(100) NOT NULL,
    ->   Email             VARCHAR(120) NOT NULL UNIQUE,
    ->   Phone             VARCHAR(20),
    ->   Address           VARCHAR(255),
    ->   Registration_Date DATE DEFAULT (CURRENT_DATE)
    -> );
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE Category (
    ->   Category_ID   INT PRIMARY KEY,
    ->   Category_Name VARCHAR(80) NOT NULL UNIQUE,
    ->   Description   VARCHAR(255)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE Product (
    ->   Product_ID     INT PRIMARY KEY,
    ->   Product_Name   VARCHAR(120) NOT NULL,
    ->   Description    VARCHAR(255),
    ->   Price          DECIMAL(10,2) NOT NULL CHECK (Price > 0),
    ->   Stock_Quantity INT NOT NULL DEFAULT 0 CHECK (Stock_Quantity >= 0),
    ->   Category_ID    INT NOT NULL,
    ->   FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID)
    -> );
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE Orders (
    ->   Order_ID     INT PRIMARY KEY,
    ->   Customer_ID  INT NOT NULL,
    ->   Order_Date   DATETIME DEFAULT CURRENT_TIMESTAMP,
    ->   Status       VARCHAR(20) NOT NULL DEFAULT 'Placed',
    ->   Total_Amount DECIMAL(10,2) NOT NULL DEFAULT 0 CHECK (Total_Amount >= 0),
    ->   CHECK (Status IN
    ->     ('Placed','Confirmed','Packed','Shipped','Delivered','Cancelled')),
    ->   FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
    -> );
Query OK, 0 rows affected (0.02 sec)
```

Figure 14.2: Execution output: Table creation (Section 6.2)

```
mysql  Ver 8.0.46  -  root@localhost : ecommerce_order_flow

mysql> CREATE TABLE Order_Item (
    ->   Order_Item_ID INT PRIMARY KEY,
    ->   Order_ID      INT NOT NULL,
    ->   Product_ID    INT NOT NULL,
    ->   Quantity      INT NOT NULL CHECK (Quantity > 0),
    ->   Unit_Price    DECIMAL(10,2) NOT NULL CHECK (Unit_Price > 0),
    ->   Subtotal      DECIMAL(10,2) NOT NULL CHECK (Subtotal >= 0),
    ->   FOREIGN KEY (Order_ID)   REFERENCES Orders(Order_ID),
    ->   FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE Payment (
    ->   Payment_ID     INT PRIMARY KEY,
    ->   Order_ID       INT NOT NULL UNIQUE,
    ->   Payment_Date   DATETIME DEFAULT CURRENT_TIMESTAMP,
    ->   Amount         DECIMAL(10,2) NOT NULL CHECK (Amount >= 0),
    ->   Payment_Method VARCHAR(20) NOT NULL,
    ->   Payment_Status VARCHAR(20) NOT NULL DEFAULT 'Pending',
    ->   Transaction_ID VARCHAR(100) UNIQUE,
    ->   CHECK (Payment_Method IN ('UPI','Card','Cash','Net Banking')),
    ->   CHECK (Payment_Status IN ('Pending','Paid','Failed','Refunded')),
    ->   FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE Shipping (
    ->   Shipping_ID      INT PRIMARY KEY,
    ->   Order_ID         INT NOT NULL UNIQUE,
    ->   Delivery_Address VARCHAR(255) NOT NULL,
    ->   Shipping_Date    DATE,
    ->   Delivery_Status  VARCHAR(20) NOT NULL DEFAULT 'Processing',
    ->   Tracking_ID      VARCHAR(100) UNIQUE,
    ->   Carrier_Name     VARCHAR(80),
    ->   CHECK (Delivery_Status IN
    ->     ('Processing','Packed','Shipped','Out for Delivery','Delivered','Returned')),
    ->   FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> SHOW TABLES;
+-------------------------------+
| Tables_in_ecommerce_order_flow |
+-------------------------------+
| Category                      |
| Customer                      |
| Order_Item                    |
| Orders                        |
| Payment                       |
| Product                       |
| Shipping                      |
+-------------------------------+
7 rows in set (0.00 sec)
```

Figure 14.3: Execution output: Table creation, continued (Section 6.2)

```
mysql  Ver 8.0.46  -  root@localhost : ecommerce_order_flow

mysql> ALTER TABLE Product ADD Brand VARCHAR(80);
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> ALTER TABLE Customer MODIFY Phone VARCHAR(25);
Query OK, 0 rows affected (0.01 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> DESC Product;
+----------------+---------------+------+-----+---------+-------+
| Field          | Type          | Null | Key | Default | Extra |
+----------------+---------------+------+-----+---------+-------+
| Product_ID     | int           | NO   | PRI | NULL    |       |
| Product_Name   | varchar(120)  | NO   |     | NULL    |       |
| Description    | varchar(255)  | YES  |     | NULL    |       |
| Price          | decimal(10,2) | NO   |     | NULL    |       |
| Stock_Quantity | int           | NO   |     | 0       |       |
| Category_ID    | int           | NO   | MUL | NULL    |       |
| Brand          | varchar(80)   | YES  |     | NULL    |       |
+----------------+---------------+------+-----+---------+-------+
7 rows in set (0.00 sec)
```

Figure 14.4: Execution output: ALTER operation and resulting Product structure (Section 6.4)

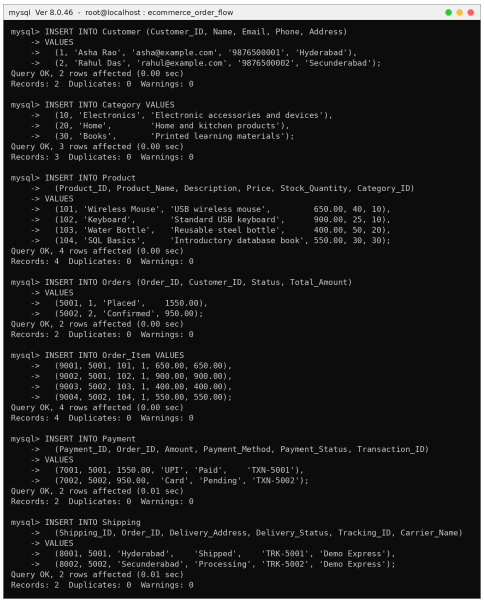


Figure 14.5: Execution output: INSERT commands (Section 7.1)

```
mysql  Ver 8.0.46  -  root@localhost : ecommerce_order_flow

mysql> UPDATE Orders SET Status = 'Packed'         WHERE Order_ID   = 5002;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE Product SET Price = 680.00           WHERE Product_ID = 101;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE Payment SET Payment_Status = 'Paid'  WHERE Payment_ID = 7002;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> DELETE FROM Shipping
    -> WHERE Shipping_ID = 8002;
Query OK, 1 row affected (0.00 sec)
```

Figure 14.6: Execution output: UPDATE and DELETE commands (Sections 7.2 and 7.3)

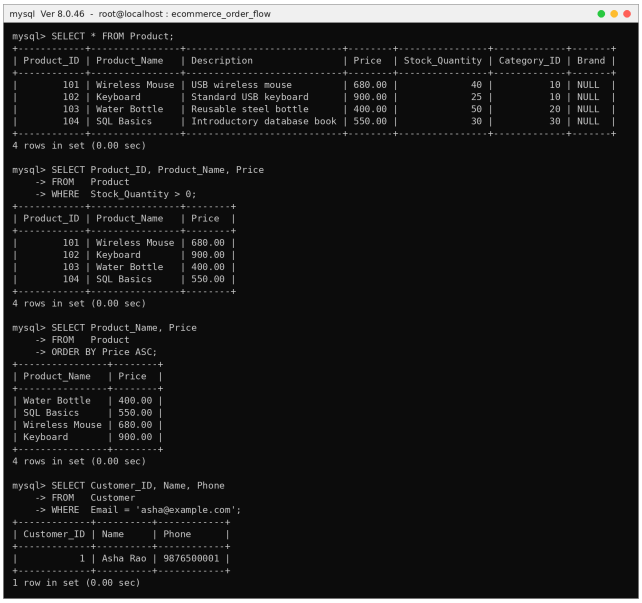


Figure 14.7: Execution output: Basic SELECT queries (Section 8.1)

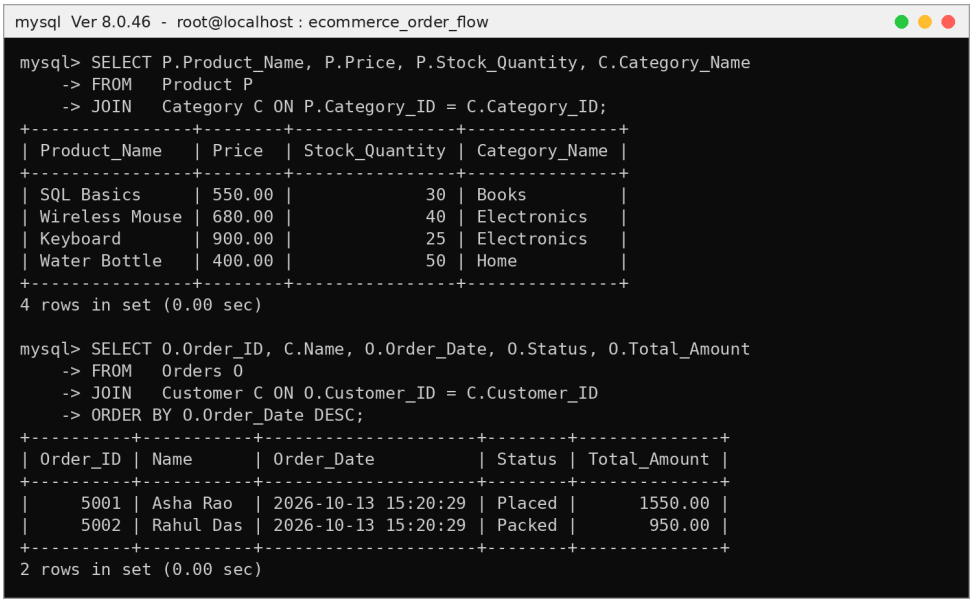


Figure 14.8: Execution output: JOIN operations: catalogue and orders with customers (Section 8.2)

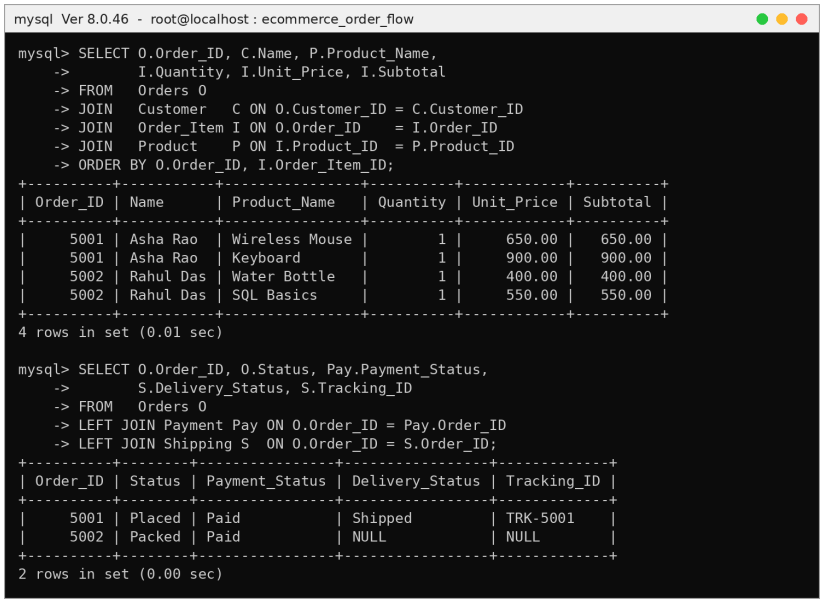


Figure 14.9: Execution output: JOIN operations: order items and payment/shipping overview (Section 8.2)

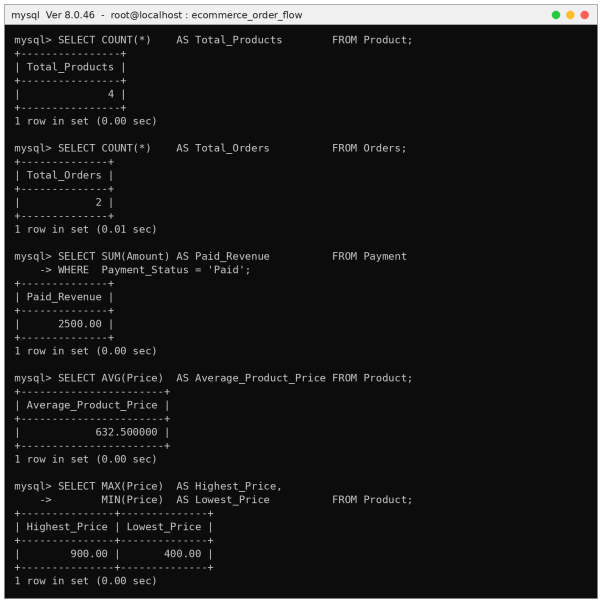


Figure 14.10: Execution output: Aggregate functions (Section 8.3)

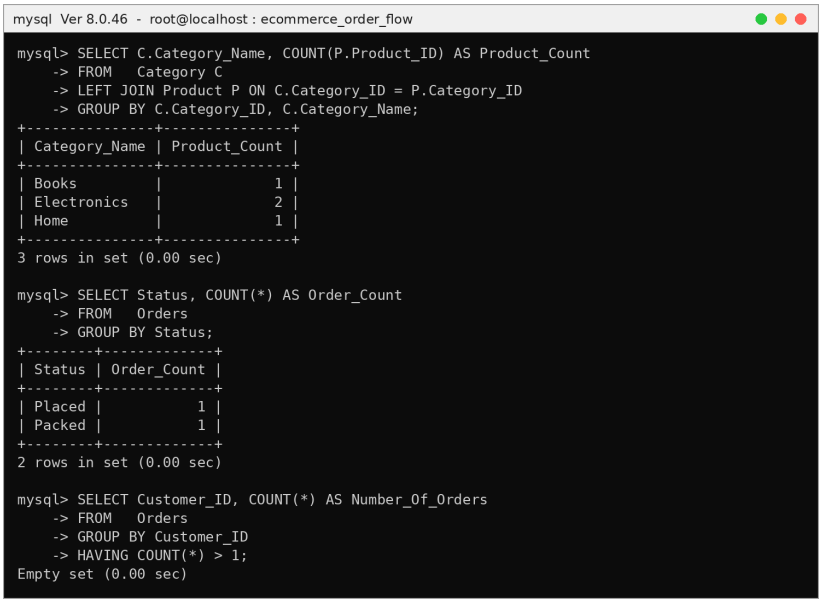


Figure 14.11: Execution output: GROUP BY and HAVING (Section 8.4)

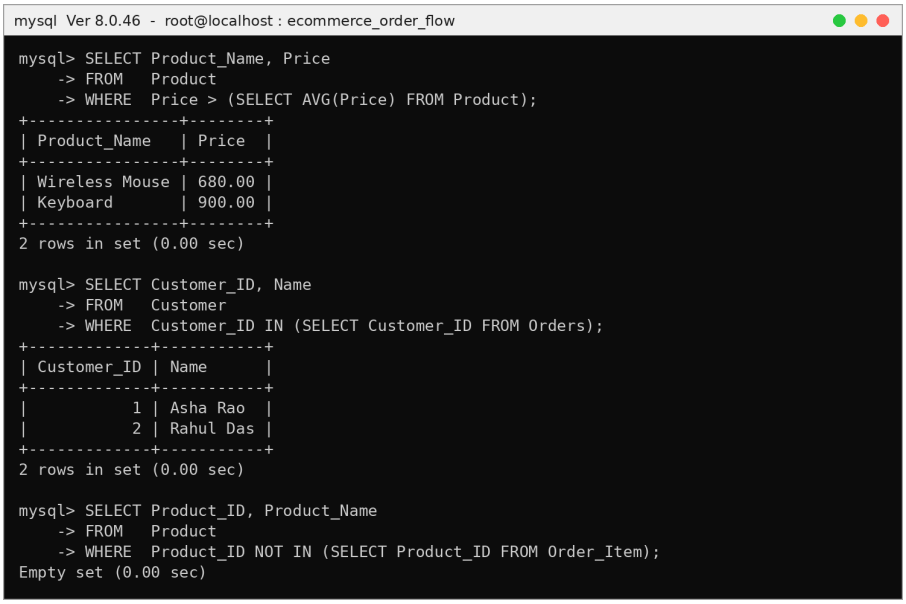


Figure 14.12: Execution output: Subqueries (Section 8.5)

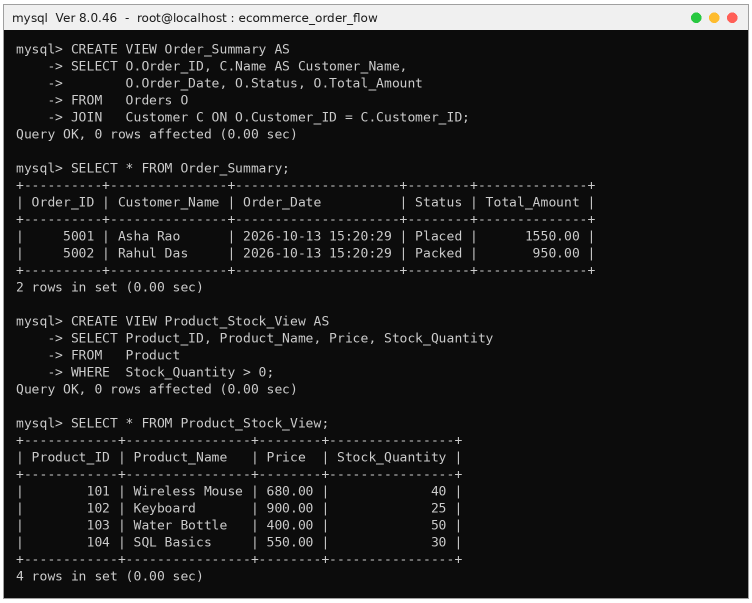


Figure 14.13: Execution output: Views (Section 8.6)

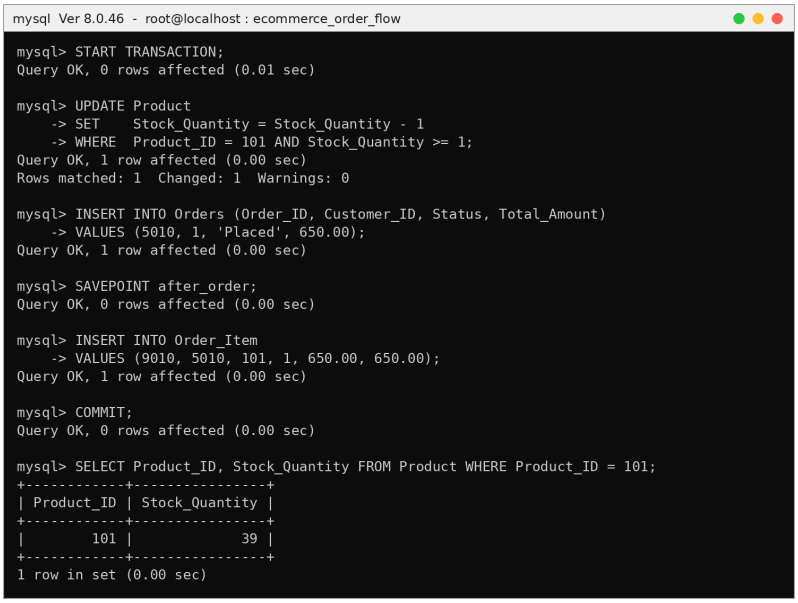


Figure 14.14: Execution output: Transaction with SAVEPOINT and COMMIT (Section 9.2)

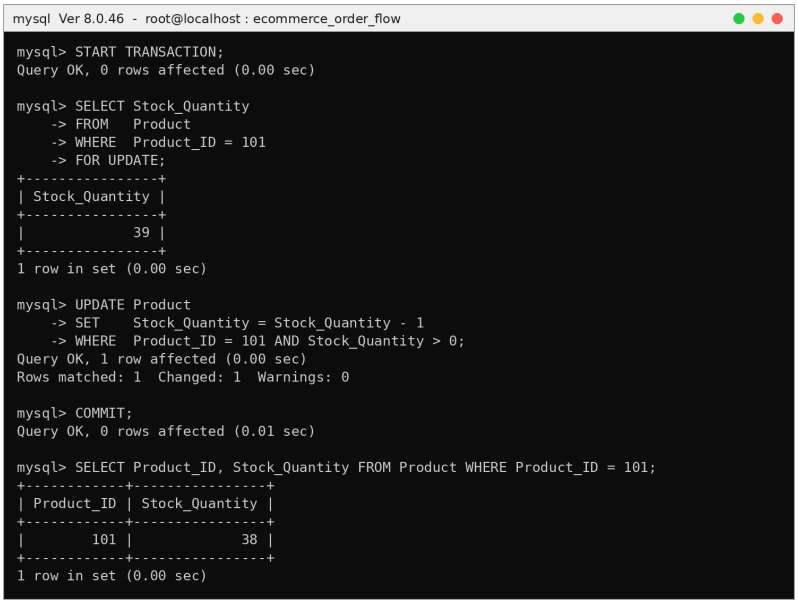


Figure 14.15: Execution output: Row locking with SELECT ... FOR UPDATE (Section 9.3)

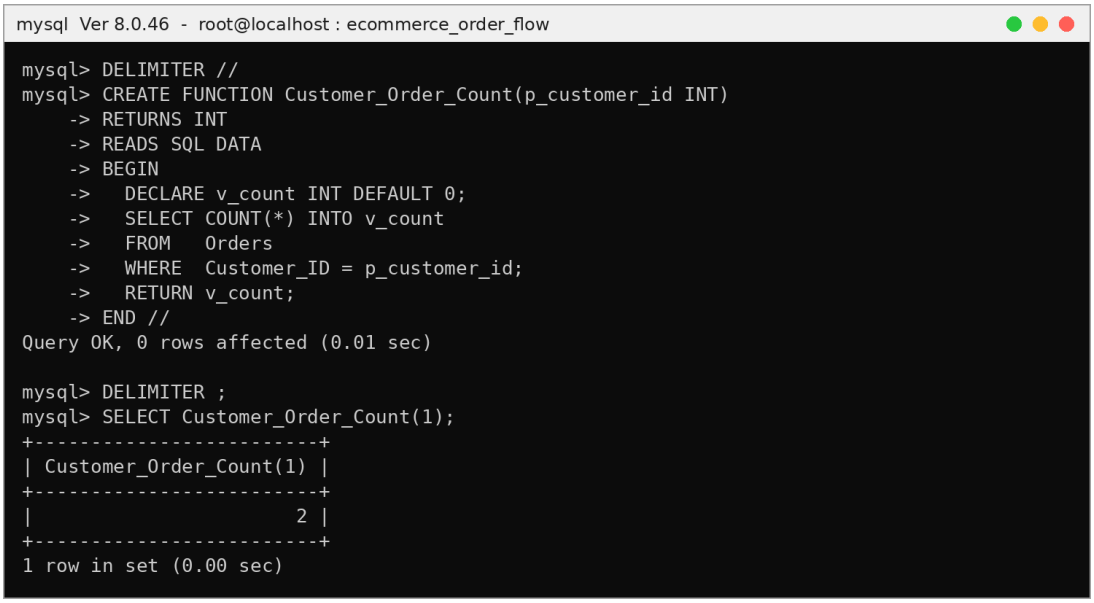


Figure 14.16: Execution output: Stored function Customer_Order_Count (Section 10.1)

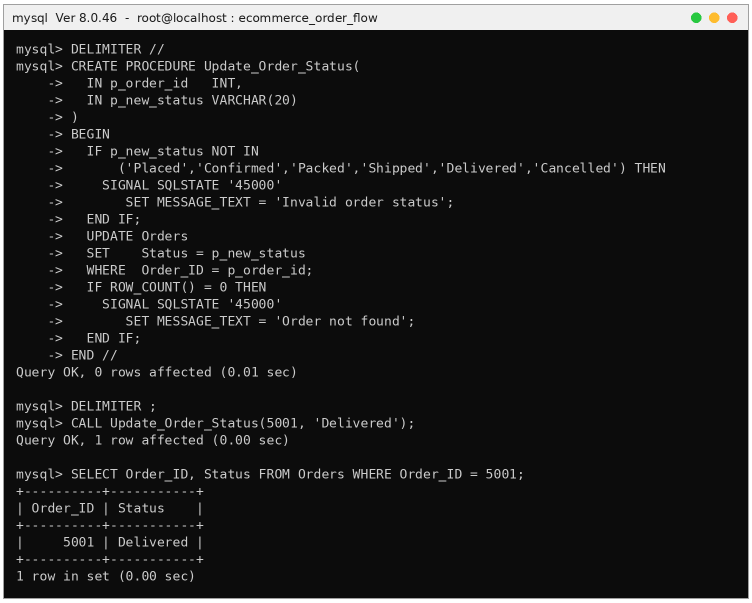


Figure 14.17: Execution output: Stored procedure Update_Order_Status (Section 10.2)

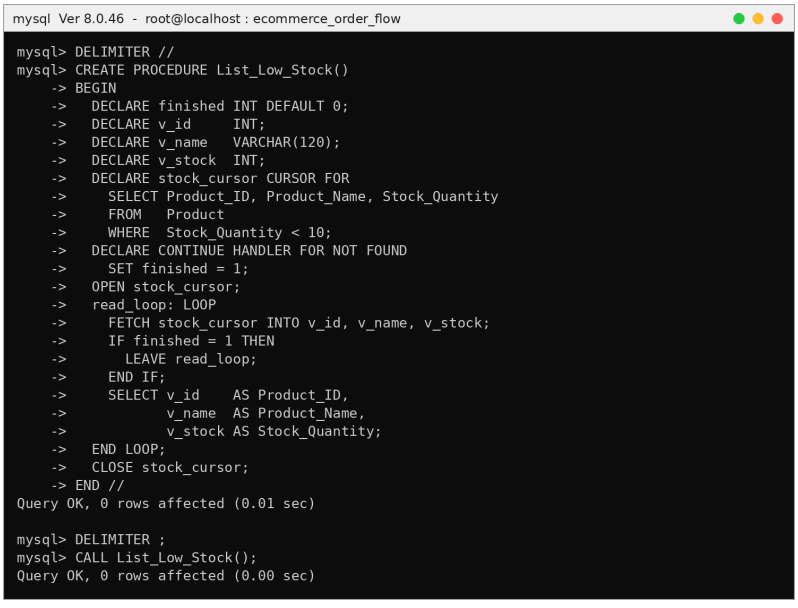


Figure 14.18: Execution output: Cursor procedure List_Low_Stock (Section 10.3)

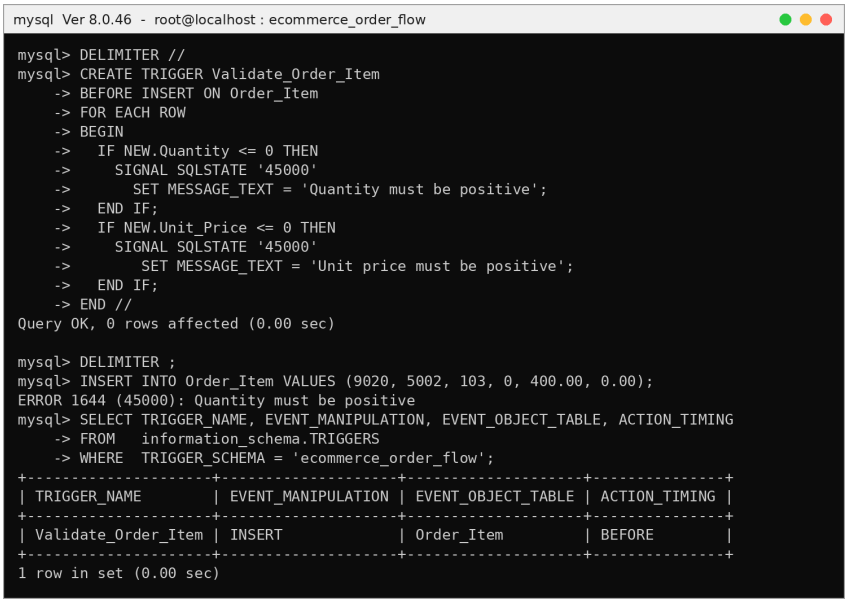


Figure 14.19: Execution output: Trigger Validate_Order_Item (Section 10.4)

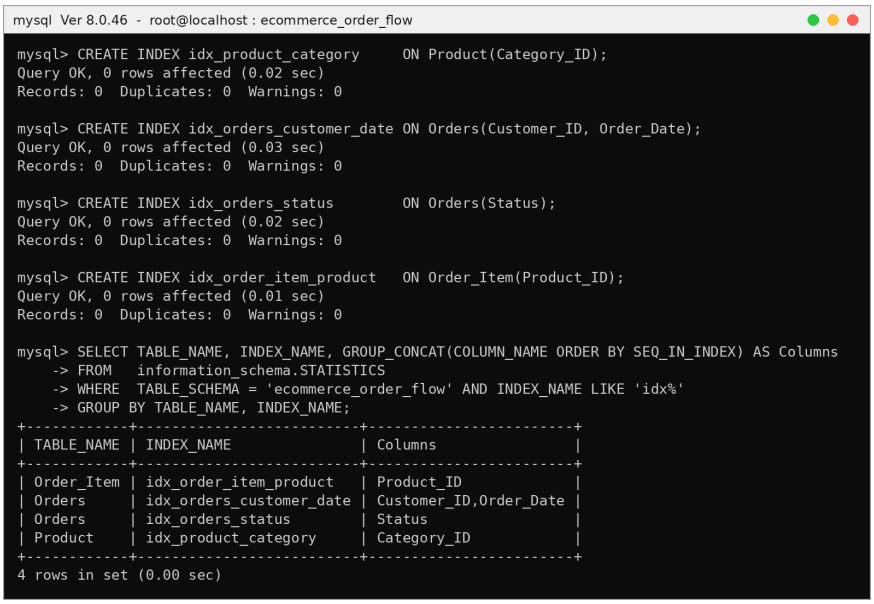


Figure 14.20: Execution output: Index creation (Section 11.2)

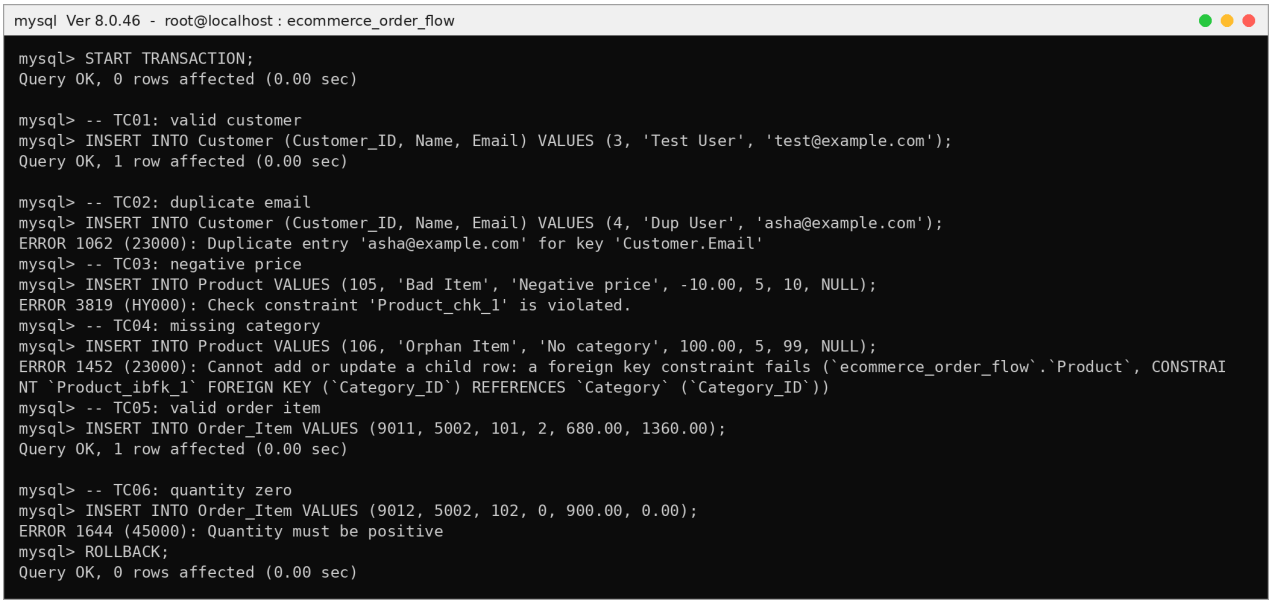


Figure 14.21: Execution output: Test cases TC01–TC06 (Section 12), run inside a transaction and rolled back

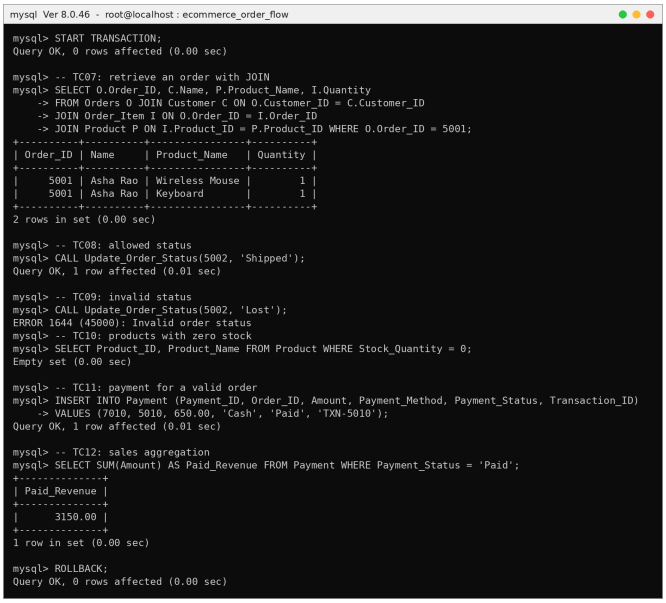


Figure 14.22: Execution output: Test cases TC07–TC12 (Section 12), run inside a transaction and rolled back

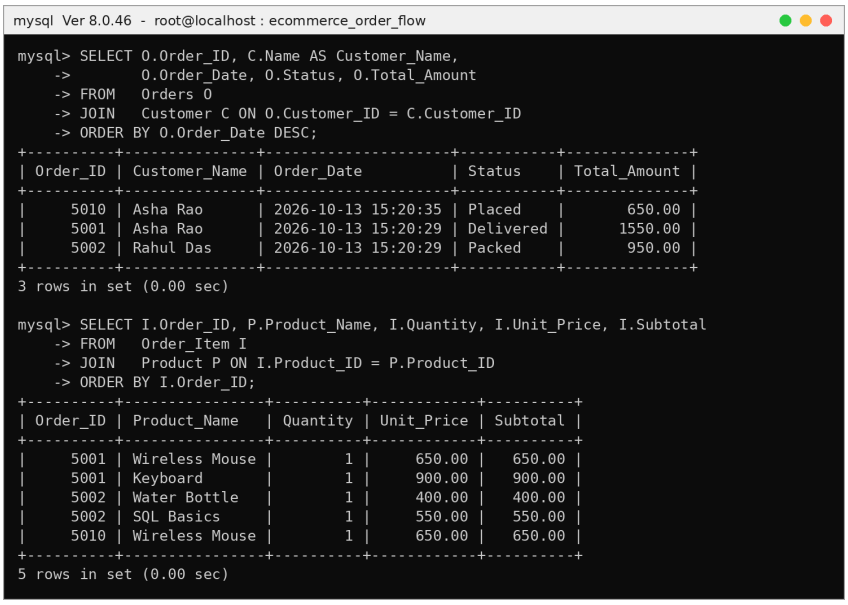


Figure 14.23: Execution output: Order report and order item details (Sections 13.1 and 13.2)

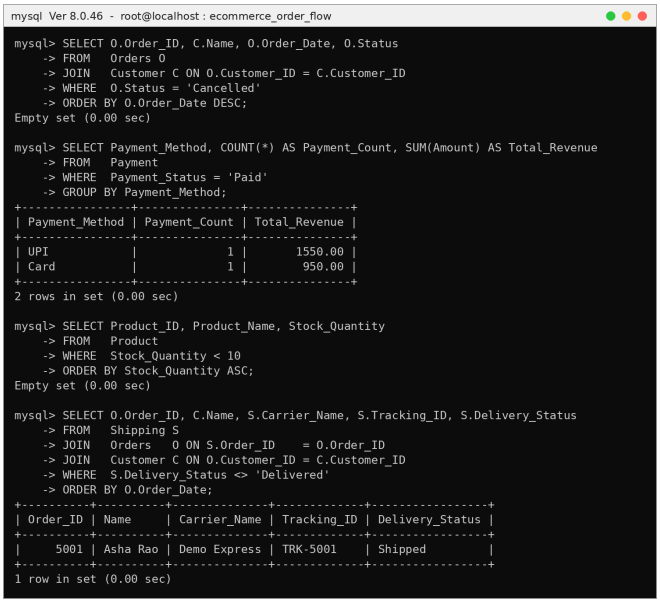


Figure 14.24: Execution output: Cancellation, revenue, inventory and shipping reports (Sections 13.3–13.6)

Test Results

Table 14.1: Test Cases for E-Commerce Order Flow (executed results)

Test ID	Test	Observed Result	Status
TC01	Insert a valid customer	Query OK, 1 row affected	Passed
TC02	Insert duplicate customer email	ERROR 1062: Duplicate entry for key `Customer.Email`	Passed
TC03	Insert a product with negative price	ERROR 3819: Check constraint `Product_chk_1` is violated	Passed
TC04	Insert product with missing category ID	ERROR 1452: foreign key constraint fails	Passed
TC05	Add a valid order item	Query OK, 1 row affected	Passed
TC06	Insert quantity zero	ERROR 1644: Quantity must be positive (trigger)	Passed
TC07	Retrieve an order with JOIN	2 rows returned with customer and product names	Passed
TC08	Update order status to an allowed value	Query OK, 1 row affected	Passed
TC09	Update order status to an invalid value	ERROR 1644: Invalid order status	Passed
TC10	Query products with zero stock	Empty set	Passed
TC11	Record payment for a valid order	Query OK, 1 row affected	Passed
TC12	Run sales aggregation	Paid_Revenue = 3150.00	Passed

Test cases were executed inside a transaction and rolled back, so they did not alter the sample data (Figures 14.21 and 14.22).

CHAPTER 15

Conclusion

The E-Commerce Order Flow project applies relational database concepts to organize customer, product, category, order, payment, and shipping information. The schema uses primary keys, foreign keys, constraints, and normalization to maintain consistent records and reduce duplication.

The SQL implementation demonstrates how records can be inserted, updated, retrieved, grouped, and reported. Transactions and indexes provide a foundation for safer order updates and efficient searches. With further testing and application integration, the database can support a more complete online order-management system.

CHAPTER 16

Mapping POs and SDGs

This chapter maps the project to selected Program Outcomes and Sustainable Development Goals. The mapping describes the relationship between the work performed and the intended outcome; it is not a claim of formal certification.

16.1 Program Outcomes (POs) Mapping

Table 16.1: Mapping of Program Outcomes to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Applied database concepts, relational modelling, and SQL operations.
PO2	Problem Analysis	Identified issues involving duplicate records, stock accuracy, order status, and payment tracking.
PO3	Design/Development of Solutions	Designed connected tables and constraints for order processing.
PO5	Modern Tool Usage	Used MySQL and SQL statements to define and query the database.
PO12	Life-long Learning	Strengthened understanding of database design, transactions, indexing, and reporting.

16.2 Sustainable Development Goals (SDGs) Mapping

Table 16.2: Mapping of SDGs to Project Relevance

SDG	Goal	Project Relevance
SDG 8	Decent Work and Economic Growth	Organized order and inventory records can support efficient business operations.
SDG 9	Industry, Innovation and Infrastructure	Demonstrates the use of digital database infrastructure for online commerce.
SDG 12	Responsible Consumption and Production	Stock visibility and accurate order records can help reduce avoidable inventory errors and waste.

The project contributes at a small, practical level by demonstrating digital information management. Its actual sustainability impact would depend on how a deployed system is used and maintained.

CHAPTER 17

Future Scope

The project can be extended in several directions:

1. **Customer authentication:** provide secure registration, login, and role-based access.
2. **Payment gateway integration:** connect to a trusted payment provider and store only appropriate transaction references.
3. **Real-time shipping updates:** integrate courier tracking APIs and update delivery status automatically.
4. **Inventory reservation:** reserve stock when an order is confirmed and release it safely when an order is cancelled.
5. **Automated notifications:** send order confirmation, payment status, dispatch, and delivery messages.
6. **Advanced analytics:** add product-wise sales, monthly revenue, order trends, and low-stock alerts.
7. **Security and auditing:** apply least-privilege database accounts, audit important changes, and protect customer data.
8. **Performance testing:** use realistic datasets and query plans to refine indexes and transaction handling.
9. **Web or mobile interface:** connect the database to a front end for customers and administrators.

These enhancements would turn the current database-focused laboratory project into a more complete order-management application.

Bibliography

[1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2016.

[2] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill, 2019.

[3] Oracle, *MySQL Reference Manual*. Available: `https://dev.mysql.com/doc/`.

[4] A. Beaulieu, *Learning SQL*, 3rd ed., O'Reilly Media, 2020.

[5] C. J. Date, *An Introduction to Database Systems*, 8th ed., Pearson, 2003.